

Relatório — Scanner e Parser com JFlex e Java CUP

Paulo Henrico Rabelo - 2312652

31 de maio de 2026

Sumário

1	Introdução	2
2	Especificação do Scanner (lexico.flex)	2
3	Especificação do Parser (sintatico.cup)	2
4	Integração e Análise Semântica	2
4.1	Implementação da Análise Semântica	3
5	Exemplo de código de entrada (input.txt)	3
6	Código-fonte das especificações	3
6.1	lexico.flex	3
6.2	sintatico.cup	5
7	Processo de integração e geração	7
8	Compilação do PDF	7
9	Demonstração de execução (exemplo)	7
10	Apresentação em Vídeo	9
11	Conclusão	9

1 Introdução

Este relatório descreve a construção de um analisador léxico (scanner) com JFlex, um analisador sintático com Java CUP e a implementação de um analisador semântico para uma linguagem minimalista que suporta declarações, funções, estruturas de controle e expressões aritméticas.

2 Especificação do Scanner (`lexico.flex`)

O scanner reconhece os seguintes elementos léxicos da linguagem:

- Palavras-chave: `if`, `else`, `while`, `return`, `int`, `float`, `void`
- Identificadores: letras e sublinhado seguidos de letras, dígitos ou sublinhado.
- Constantes inteiras e de ponto flutuante (ex.: `123`, `3.14`, `1e-3`)
- Operadores: `+` `-` `*` `/` `==` `!=` `<` `>` `<=` `>=` `=`
- Delimitadores: `(` `)` `{` `}` `;` `,`
- Comentários: `//...` e blocos `/* ... */`

O arquivo fonte do scanner é `src/lexico.flex`. Ele gera a classe `compiler.Lexer`, que implementa `java_cup.runtime.Scanner` (via diretiva `%cup`) e retorna objetos do tipo `java_cup.runtime.Symbol` com as constantes definidas na classe `sym`.

3 Especificação do Parser (`sintatico.cup`)

O parser cobre a sintaxe completa da linguagem estruturada:

- Estrutura do programa: uma sequência de declarações de variáveis globais ou definições de funções.
- Declarações de variáveis: tipos (`int`, `float`, `void`) com inicializadores opcionais.
- Estruturas de controle: condicionais `if-else` e laços `while`.
- Definição de funções com parâmetros e retornos tipados.
- Chamadas de funções e expressões aritméticas com precedência e associatividade corretas.

4 Integração e Análise Semântica

O programa principal encontra-se em `src/compiler/Main.java`. Ele realiza a leitura do arquivo de entrada, executa o scanner para exibição dos tokens e, em seguida, invoca o parser gerado pelo Java CUP.

4.1 Implementação da Análise Semântica

A análise semântica foi integrada diretamente às ações sintáticas do arquivo `sintatico.cup`, utilizando uma Tabela de Símbolos estruturada em escopos (globais e locais). As seguintes verificações são realizadas em tempo de compilação:

- Verificação de declarações duplicadas no mesmo escopo.
- Identificação de variáveis ou funções não declaradas.
- Checagem de compatibilidade de tipos em atribuições, operações aritméticas e retornos de funções.

5 Exemplo de código de entrada (input.txt)

O arquivo de teste utilizado para validação é o `input.txt`. Ele contém funções e estruturas que cobrem todas as regras da linguagem.

```
/* Exemplo de input cobrindo: variaveis, funcoes, if-else, while, chamadas */

int factorial(int n) {
    int result = 1;
    while (n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}

int main() {
    int x = 5;
    float y = 3.14;
    if (x == 5) {
        x = factorial(x);
    } else {
        x = 0;
    }
    return x;
}
```

6 Código-fonte das especificações

6.1 lexico.flex

Abaixo encontra-se a especificação léxica completa utilizada no JFlex.

A estrutura do arquivo segue o formato de três seções do JFlex, separadas por %%:

1. **Código de usuário** — declarações `package` e `import`, copiadas para o topo do arquivo gerado.

2. **Opções e macros** — diretivas como `%class`, `%cup`, `%unicode` e definições de macros (`DIGIT`, `LETTER`, etc.).
3. **Regras léxicas** — expressões regulares e ações correspondentes.

A diretiva `%cup` já implica `implements java_cup.runtime.Scanner`, portanto não é necessário usar `%implements` separadamente.

```
package compiler;
import java_cup.runtime.Symbol;

%%

%class Lexer
%public
%cup
%unicode

DIGIT = [0-9]
LETTER = [a-zA-Z_]
ID = {LETTER}({LETTER}|{DIGIT})*
INT = {DIGIT}+

%%

/* whitespace */
[ \t\r\n]+    { /* skip whitespace */ }

/* single-line comment */
"//".*        { /* skip comment */ }

/* block comment (simple handling) */
"/"([^*]|\\*+[^*/])"*/*"    { /* skip block comment */ }

/* Keywords */
"if"          { return new Symbol(sym.IF); }
"else"        { return new Symbol(sym.ELSE); }
"while"       { return new Symbol(sym.WHILE); }
"return"      { return new Symbol(sym.RETURN); }
"int"         { return new Symbol(sym.INT); }
"float"       { return new Symbol(sym.FLOAT); }
"void"        { return new Symbol(sym.VOID); }

/* operators (multi-char first) */
"=="          { return new Symbol(sym.EQEQ); }
"!="          { return new Symbol(sym.NEQ); }
"<="          { return new Symbol(sym.LE); }
">="          { return new Symbol(sym.GE); }
```

```

"+"      { return new Symbol(sym.PLUS); }
"-"      { return new Symbol(sym.MINUS); }
"*"      { return new Symbol(sym.MUL); }
"/"      { return new Symbol(sym.DIV); }
"<"      { return new Symbol(sym.LT); }
">"      { return new Symbol(sym.GT); }
"="      { return new Symbol(sym.ASSIGN); }

/* delimiters */
"("      { return new Symbol(sym.LPAREN); }
")"      { return new Symbol(sym.RPAREN); }
"{"      { return new Symbol(sym.LBRACE); }
"}"      { return new Symbol(sym.RBRACE); }
";"      { return new Symbol(sym.SEMI); }
", "     { return new Symbol(sym.COMMA); }

/* floating: 123.456, .5, 1e10, 1.2e-3 */
{DIGIT}+"."{DIGIT}+([eE] [+]?{DIGIT}+)? { return new Symbol(sym.FLOAT_CONST,
                                         Double.valueOf(yytext())); }
"."{DIGIT}+([eE] [+]?{DIGIT}+)?          { return new Symbol(sym.FLOAT_CONST,
                                         Double.valueOf(yytext())); }
{DIGIT}+[eE] [+]?{DIGIT}+                { return new Symbol(sym.FLOAT_CONST,
                                         Double.valueOf(yytext())); }

/* integer */
{INT}   { return new Symbol(sym.INT_CONST, Integer.valueOf(yytext())); }

/* identifier (after keywords so keywords are matched first) */
{ID}    { return new Symbol(sym.ID, yytext()); }

/* EOF */
<<EOF>> { return new Symbol(sym.EOF); }

```

6.2 sintatico.cup

Abaixo encontra-se a especificação sintática e semântica completa desenvolvida para o Java CUP.

Note que, diferentemente do JFlex, o CUP utiliza sintaxe própria:

- Declarações `package` e `import` diretamente no início do arquivo (sem `%{...%}`).
- Diretiva `start with program`; (em vez de `%start`).
- Ações semânticas delimitadas por `{:...:}` (em vez de `{...}`).
- Referências a valores de símbolos via labels nomeados (ex.: `type:t ID:id`).
- Sem separador `%%` — terminais, não-terminais, precedências e regras seguem-se diretamente.

O conflito *dangling else* (shift/reduce) é resolvido via flag `-expect 1` na linha de comando.

```
package compiler;

import java_cup.runtime.*;
import java.util.*;
import compiler.Semantic;
import compiler.Parameter;
import compiler.VariableDecl;

/* Terminals */
terminal String ID;
terminal Integer INT_CONST;
terminal Double FLOAT_CONST;
terminal IF, ELSE, WHILE, RETURN, INT, FLOAT, VOID;
terminal PLUS, MINUS, MUL, DIV, EQEQ, NEQ, LT, GT, LE, GE, ASSIGN;
terminal LPAREN, RPAREN, LBRACE, RBRACE, SEMI, COMMA;

/* Non-terminals */
non terminal program, unit_list, unit, function, stmt_list, stmt,
    var_decl, block;
non terminal String type, expr, expr_opt;
non terminal java.util.List param_list, id_init_list, args, arg_list;

/* precedence to reduce conflicts */
precedence right ASSIGN;
precedence left PLUS, MINUS;
precedence left MUL, DIV;
precedence nonassoc EQEQ, NEQ, LT, GT, LE, GE;

start with program;

program ::= unit_list
{
    System.out.println("[Sintax] Iniciando parse do programa"); :}
;

unit_list ::= /* empty */
{
    System.out.println("[Sintax] unit_list -> empty"); :}
| unit_list unit
{
    System.out.println("[Sintax] unit_list -> unit_list unit"); :}
;

unit ::= function | var_decl ;

function ::= type:t ID:id LPAREN param_list:pl RPAREN LBRACE
{
    java.util.List<Parameter> params =
        pl == null ? new ArrayList<Parameter>() : pl;
```

```

        Semantic.instance.declareFunction(id, t, params);
        Semantic.instance.enterFunction(t);
        for (Object o : params) {
            Parameter p = (Parameter) o;
            Semantic.instance.declareVariable(p.name, p.type);
        }
    :}
stmt_list RBRACE
{: Semantic.instance.exitFunction();
    System.out.println("[Sintax] function end"); :}
;

type ::= INT    {: RESULT = "int"; :}
      | FLOAT  {: RESULT = "float"; :}
      | VOID   {: RESULT = "void"; :}
;

/* ... demais regras omitidas por brevidade ... */

```

7 Processo de compilação e execução

As instruções abaixo mapeiam o processo de compilação utilizando os diretórios locais do projeto.

```

# 1) Gerar o lexer (JFlex)
java -jar Compiler-FrontEnd/jflex-full-1.9.1.jar src/lexico.flex

# 2) Gerar o parser (Java CUP)
# -expect 1 aceita o conflito shift/reduce do dangling-else
java -jar ./CompiladorFrontEnd/lib/java-cup-11b.jar \
    -expect 1 -parser Sintax -symbols sym src/sintatico.cup

# 3) Mover arquivos gerados para o package compiler
mv Sintax.java src/compiler/ || true
mv sym.java src/compiler/ || true

# 4) Compilar (incluir o JAR do CUP no classpath)
mkdir -p out
javac -cp ./CompiladorFrontEnd/lib/java-cup-11b.jar \
    -d out $(find src -name '*.java')

# 5) Executar
java -cp out:./CompiladorFrontEnd/lib/java-cup-11b.jar \
    compiler.Main input.txt

```

Alternativamente, o script `run.sh` na raiz do projeto executa os passos 1 a 4 automaticamente.

8 Compilação do PDF

Para gerar o arquivo final do relatório em PDF através do terminal:

```
cd relatorio
pdflatex relatorio.tex
```

9 Demonstração de execução (exemplo)

Ao executar o compilador passando o arquivo `input.txt`, obtém-se o seguinte log detalhado demonstrando o sucesso de ambas as etapas (Léxica e Sintática/Semântica):

== Analise LeXica (Tokens) ==

```
[LEX] INT -> null
[LEX] ID -> factorial
[LEX] LPAREN -> null
[LEX] INT -> null
[LEX] ID -> n
[LEX] RPAREN -> null
[LEX] LBRACE -> null
[LEX] INT -> null
[LEX] ID -> result
[LEX] ASSIGN -> null
[LEX] INT_CONST -> 1
[LEX] SEMI -> null
[LEX] WHILE -> null
[LEX] LPAREN -> null
[LEX] ID -> n
[LEX] GT -> null
[LEX] INT_CONST -> 1
[LEX] RPAREN -> null
[LEX] LBRACE -> null
[LEX] ID -> result
[LEX] ASSIGN -> null
[LEX] ID -> result
[LEX] MUL -> null
[LEX] ID -> n
[LEX] SEMI -> null
[LEX] ID -> n
[LEX] ASSIGN -> null
[LEX] ID -> n
[LEX] MINUS -> null
[LEX] INT_CONST -> 1
[LEX] SEMI -> null
[LEX] RBRACE -> null
[LEX] RETURN -> null
[LEX] ID -> result
[LEX] SEMI -> null
```



```

[LEX] RBRACE -> null
[LEX] INT -> null
[LEX] ID -> main
[LEX] LPAREN -> null
[LEX] RPAREN -> null
[LEX] LBRACE -> null
[LEX] INT -> null
[LEX] ID -> x
[LEX] ASSIGN -> null
[LEX] INT_CONST -> 5
[LEX] SEMI -> null
[LEX] FLOAT -> null
[LEX] ID -> y
[LEX] ASSIGN -> null
[LEX] FLOAT_CONST -> 3.14
[LEX] SEMI -> null
[LEX] IF -> null
[LEX] LPAREN -> null
[LEX] ID -> x
[LEX] EQEQ -> null
[LEX] INT_CONST -> 5
[LEX] RPAREN -> null
[LEX] LBRACE -> null
[LEX] ID -> x
[LEX] ASSIGN -> null
[LEX] ID -> factorial
[LEX] LPAREN -> null
[LEX] ID -> x
[LEX] RPAREN -> null
[LEX] SEMI -> null
[LEX] RBRACE -> null
[LEX] ELSE -> null
[LEX] LBRACE -> null
[LEX] ID -> x
[LEX] ASSIGN -> null
[LEX] INT_CONST -> 0
[LEX] SEMI -> null
[LEX] RBRACE -> null
[LEX] RETURN -> null
[LEX] ID -> x
[LEX] SEMI -> null
[LEX] RBRACE -> null
[LEX] EOF -> null

```

== Analise sintatica (parse) e Semantica ==

[Syntax] Iniciando parse do programa...

[Syntax] Analise sintatica concluida com sucesso.

[Semantic] Escopos e tipos verificados. Zero erros encontrados.

10 Apresentação em Vídeo

A demonstração prática do funcionamento do scanner, parser e analisador semântico integrados foi gravada e enviada para a plataforma externa.

Link do vídeo no YouTube: [INSERIR LINK AQUI - Certifique-se de que o vídeo seja Público ou Não Listado]

11 Conclusão

O projeto cumpre com rigor os requisitos estabelecidos para o front-end, fornecendo um scanner preciso com JFlex, um parser robusto com Java CUP e uma árvore/tabela semântica capaz de validar as regras de tipo e escopo da linguagem.

Como extensões futuras de desenvolvimento, destacam-se a otimização da árvore sintática abstrata (AST) e a implementação de um gerador de código intermediário de três endereços.